

Imperial College London

MENG INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Project Title

Author:
John Smith

Supervisor:
Prof. Lily Smith

Second Marker:
Dr. Adam Smith

January 20, 2026

Abstract

Your abstract goes here

Acknowledgements

Thanks mum!

Contents

1	Introduction	5
1.1	Background	5
1.2	Motivation	5
1.3	Research Objectives	6
1.4	Approach and Scope	6
2	Background	8
3	PROJECT X	9
4	Evaluation	10
5	Conclusion	11
A	First Appendix	12

List of Figures

1.1	Illustration of refactoring trajectories scored by process-quality metric J : the solid line shows the observed developer trajectory, while the dashed line shows a higher-scoring reference trajectory that diverges at an intermediate state and converges near the end.	7
-----	--	---

List of Tables

Chapter 1

Introduction

1.1 Background

Refactoring is a routine activity in software development: developers restructure code to improve readability, maintainability, and evolvability while intending not to change externally observable behaviour. Classic definitions emphasise behaviour preservation and describe refactoring as a sequence of small, semantics-preserving transformations (Opdyke, 1992; Fowler, 1999/2018), and subsequent surveys systematise these refactoring operators and the surrounding tool support (Mens & Tourwé, 2004). In practice, refactoring is widely encouraged because poorly structured code becomes an obstacle to enhancement and maintenance, and continual adaptation is necessary to keep evolving systems satisfactory (e.g., the motivation articulated in work building on Lehman’s laws and empirical maintenance observations). At the same time, refactoring remains costly: it consumes developer time, introduces risk, and is frequently intertwined with other work such as feature development and bug fixing rather than occurring as isolated “refactoring-only” episodes (Murphy-Hill et al., 2008/2009; Kim et al., 2012).

1.2 Motivation

Most research and tooling that evaluates refactoring focuses on *outcomes* rather than the *process*. A common paradigm is to treat refactoring success as a change in static quality indicators between a “before” and “after” snapshot: reductions in code smells, improvements in cohesion/coupling, decreases in complexity, or related maintainability proxies. Representative threads include metric-driven detection or prioritisation of candidate refactorings (e.g., using metric deltas to distinguish refactoring-like changes from functional modifications, or to select among refactoring options), as well as refactoring recommendation systems that rank alternatives by their impact on design metrics. More recent work expands the signals used for guidance—incorporating traceability information alongside code metrics when ranking refactoring solutions, and exploring how product/process metrics relate to developers’ motivations to refactor (e.g., studies building on mined refactorings and repository history, and proposals that use LLMs to infer motivations from commit artefacts). In parallel, a substantial line of work has improved the ability to *detect* refactorings from code changes, notably by using AST-based differencing and structured matching to infer refactoring operations between revisions (e.g., RefactoringMiner and related approaches), enabling large-scale empirical analyses of refactoring frequency, types, and context (Tsantalis et al., 2018/2020; Falleri et al., 2014).

However, these perspectives largely evaluate refactoring as a mapping from an initial program state to a final program state. They do not directly address whether the *trajectory* taken to reach the end state was efficient, low-risk, or “good” by any explicit criterion. This is an important omission because developers rarely refactor in a single atomic operation: they work through intermediate states, sometimes detour, sometimes perform rework, and often create temporary degradation (e.g., in readability, modular structure, or test/build stability) before arriving at a final outcome. Two developers can produce the same end state via very different paths—one might take many disruptive steps, oscillate between designs, or introduce avoidable churn; another might progress through a more stable sequence of changes. Existing endpoint-based evaluation cannot

distinguish these cases. Meanwhile, research on process realism suggests that refactoring is frequently “flossed” into ongoing work rather than performed as a separated “root canal” activity, increasing the likelihood of mixed-intent steps and noisy traces (Murphy-Hill et al., 2008/2009). This makes the quality of the process—how a developer navigates intermediate states—potentially consequential for both developer productivity and software health during evolution.

1.3 Research Objectives

This project investigates the following thesis question: **given a known start state and a known end state, can we evaluate the quality of the developer’s refactoring process, and identify points where a better process was available?** The core idea is to explicitly model refactoring as a *process* over states rather than only an endpoint comparison. Concretely, the project adopts a state/action/trace framing: snapshots of the codebase are treated as *states*, and developer edits—including IDE refactorings and manual transformations—are treated as *actions* that produce a trace from the start state to the end state. A central component will be a **process-quality metric** that scores traces (and potentially intermediate steps) according to criteria that will be defined and justified using the literature on code quality metrics, smell detection validity, and process signals (e.g., DECOR-style smell detection; empirical findings on metric reliability and disagreement; and work linking process/product factors to refactoring likelihood and developer intent). The metric is not assumed to be purely structural: it may incorporate multiple terms capturing endpoint improvement, intermediate-state degradation, churn/disruption, and “safety” proxies such as build/test preservation, reflecting known limitations of relying on any single signal.

To move beyond scoring a single observed trace, the project will also explore **counterfactual alternatives**: at selected points along the developer’s trace, generate plausible alternative next steps (or short-horizon paths) and compare their scores to the developer’s choice. This aligns with, but differs from, prior work that synthesises or recommends refactoring sequences to reach a target design. For example, ReSynth synthesises sequences of IDE refactorings consistent with a developer’s partial edits to complete a transformation (Raychev et al., 2013), while Refactoring Navigator suggests refactoring steps to guide a system toward a desired architectural target (Lin et al., 2016). These tools demonstrate the feasibility of generating refactoring sequences and navigating toward goals, yet they are not designed to evaluate the quality of a human-chosen process given a user-defined metric. In this project, counterfactual generation is used specifically to support process evaluation—surfacing “you could have done X here” comparisons—rather than to replace the developer’s work or to optimise solely for reachability of the end state.

The expected outcome is an **offline analysis tool** that takes (i) a repository, (ii) a start and end revision, and (iii) an observed trace (e.g., commits or checkpoints), and produces a process-quality assessment: an overall score, step-level breakdown, and highlighted divergence points with higher-scoring local alternatives. IDE integration is considered an extension rather than a requirement for the interim stage; nevertheless, related work on IDE interaction logging and refactoring activity traces suggests that richer traces are feasible under appropriate privacy constraints, and may later support finer-grained “step” definitions than commit boundaries.

1.4 Approach and Scope

The proposed approach targets real-world refactoring as it occurs in development, including sequences that involve both IDE-supported refactorings and manually performed edits. The initial focus is on a single language ecosystem (e.g., Java) to leverage mature parsing infrastructure, established quality metrics, and state-of-the-art refactoring mining (e.g., RefactoringMiner). A refactoring process is represented as a sequence of steps between snapshots; in the first instance, steps may correspond to commits or developer-defined checkpoints, and this representation can be refined using higher-granularity IDE interaction data when available.

The system is intended to comprise (i) an IDE plugin for capturing refactoring traces and presenting feedback, and (ii) an analysis component that scores the observed refactoring process and constructs higher-scoring alternative trajectories between the same start and end states under a chosen process-quality metric. Rather than attempting to enumerate all possible refactoring operators or to compute globally optimal plans, the goal is to produce a plausible, metric-improving reference trajectory and to explain the principal points at which the developer’s trajectory diverged

from it (e.g., where an earlier restructuring could have avoided churn, reduced temporary quality degradation, or preserved build/test stability).

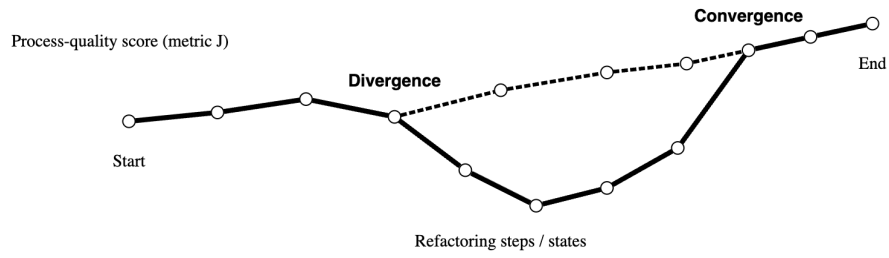


Figure 1.1: Illustration of refactoring trajectories scored by process-quality metric J : the solid line shows the observed developer trajectory, while the dashed line shows a higher-scoring reference trajectory that diverges at an intermediate state and converges near the end.

Chapter 2

Background

Chapter 3

PROJECT X

Chapter 4

Evaluation

Chapter 5

Conclusion

Appendix A

First Appendix

Bibliography